VisionGuard AI — Presentation Guide

Use this as a 5–7 minute speaking guide with a live demo.

Opening — 20 seconds

VisionGuard AI is a visual defect inspection platform built to demonstrate both machine-learning reasoning and software-engineering delivery. The demo is local and reproducible, so I can explain every step without depending on a paid service.

Problem — 40 seconds

Manual visual inspection is slow and inconsistent. VisionGuard demonstrates an inference service that identifies surface anomalies, localizes them, scores confidence/severity and exposes the result through a browser dashboard.

Architecture — 60 seconds

Draw or explain the pipeline from input/data source to model/retrieval logic, then to the FastAPI service and dashboard. Emphasize the separation between model logic and product/API logic.

Live Demo — 2 to 3 minutes

Follow the numbered demo flow below. Pause at the algorithmic output and explain what the model/system is doing rather than clicking quickly through screens.

- Step 1: Start with the Scratch example that loads automatically.
- Step 2: Point out the localized bounding box and confidence score.
- Step 3: Switch to Normal, Surface Spot and Edge Chip.
- Step 4: Upload a custom image and show the same API path.
- Step 5: Explain the trained-model upgrade path with PyTorch/YOLO, versioned weights and GPU inference.

Engineering — 45 seconds

Mention validation, tests, Docker, CI and the reason the demo can run locally. Then identify the optional components that would be used in a larger deployment.

Close — 30 seconds

Summarize one technical result, one design trade-off and one next improvement. Invite questions about model choice, metrics, scale or failure handling.

Resume-ready bullets

- Built a computer-vision inspection service that detects surface defects, localizes anomalous regions with bounding boxes, and returns confidence/severity scores through a FastAPI inference API.
- Implemented image preprocessing, region-of-interest analysis, real-time browser visualization, sample inspection workflows, request validation, Docker deployment and automated tests.
- Added pluggable PyTorch/YOLO model adapters so the lightweight local demo can be upgraded to trained deep-learning detection models without changing the API contract.

Metrics to mention if asked

- Precision/Recall/F1
- mAP50 / mAP50-95
- IoU localization
- False reject rate
- Inference latency

30-second fallback if the live demo fails

> *“The application normally runs locally through FastAPI. I also keep a saved deterministic output and automated tests in the repository. I will show the expected output first, then walk through the code path that produces it and the test that verifies the behavior.”*

Rapid Interview Q&A;

Which metrics would you report?

Precision, recall, F1, per-class confusion matrix, mAP50/mAP50-95, localization IoU, inference latency and false-reject rate.

How do you handle class imbalance?

Targeted augmentation, weighted loss/focal loss, balanced sampling and threshold tuning per defect class.

Before the interview

- Run the app once from a clean terminal.
- Run `pytest -q` and confirm all tests pass.
- Know the main API endpoint and one representative request/response.
- Know one limitation and one scale-up plan.
- Do not claim optional technology as trained/deployed unless you have actually run it.